

Содержание

1. Назначение сервиса
2. Общая схема
3. Технологии
4. Пакет `ru.adnr.flowmanager`
5. Пакет `controller`
6. Пакет `facade`
7. Пакет `dto`
8. Пакет `service`
9. Пакет `service.impl`
10. Пакет `entity`
11. Пакет `repository`
12. Пакет `kafka`
13. Пакет `config`
14. Пакет `exception`
15. Схема PostgreSQL
16. Полный пользовательский сценарий
17. Что решает outbox pattern
18. Конфигурация по умолчанию
19. Тесты
20. Основные достоинства
21. Проблемы и риски
22. Рекомендуемые улучшения

Подробная документация сервиса `flow-manager`

1. Назначение сервиса

`flow-manager` — REST-вход в асинхронный процесс конвертации документов. Он принимает файл от клиента, сохраняет оригинал в MinIO, создает задачу в PostgreSQL, надежно отправляет запрос конвертеру через Kafka, принимает результат и позволяет клиенту проверить статус или скачать готовый PDF.

Сервис решает следующие проблемы:

- скрывает Kafka и MinIO за простым HTTP API;
- не заставляет HTTP-клиента ждать завершения конвертации;
- хранит состояние длительной операции;
- не теряет запрос между записью задачи и отправкой Kafka-события благодаря outbox pattern;
- повторяет неуспешную отправку в Kafka;
- связывает ответ конвертера с исходной задачей;
- централизованно возвращает понятные HTTP-ошибки.

2. Общая схема



3. Технологии

- Java 17;
- Spring Boot 3.5.0;
- Spring MVC;
- Spring Validation;
- Spring Kafka;
- Spring Data JPA и Hibernate;
- PostgreSQL 16;
- Liquibase;
- MinIO;
- Eureka Client;
- Maven, JUnit 5 и Mockito.

4. Пакет `ru.adnr.flowmanager`

`FlowManagerApplication`

Точка запуска Spring Boot. `main` создает application context. `@EnableDiscoveryClient` регистрирует сервис в Eureka под именем `flow-manager`.

Класс не содержит бизнес-логики, но определяет корень component scan для всех вложенных пакетов.

5. Пакет `controller`

`FileController`

REST-контроллер с базовым путем `/api/v1/files`.

`POST /api/v1/files` принимает `multipart/form-data`, часть `file`, и возвращает `FileUploadResponse`. Контроллер передает файл в facade, где проверяется пустое содержимое.

`GET /api/v1/files/{id}/status` возвращает состояние задачи, путь результата и ошибку.

`GET /api/v1/files/{id}/download` возвращает готовый PDF как attachment. Имя задается через `Content-Disposition`, тип — `application/pdf`, содержимое передается потоково через `InputStreamResource`.

Контроллер решает задачу преобразования HTTP-запросов в вызовы прикладного слоя и не содержит правил хранения или Kafka.

6. Пакет `facade`

`FileFlowFacade`

Тонкий слой между controller и service. Метод `upload` проверяет `MultipartFile.isEmpty()` и выбрасывает `EmptyFileException`, затем делегирует обработку `FileFlowService`.

Facade отделяет входную проверку HTTP-файла от основного сценария. Сейчас он небольшой, но является местом для дополнительных правил приема: допустимые расширения, MIME type, антивирусная проверка и бизнес-лимиты.

7. Пакет `dto`

Все DTO реализованы как records и являются неизменяемыми.

`FileUploadResponse`

Ответ после приема файла:

- `fileId` — UUID созданной задачи;
- `status` — начальное состояние `PROCESSING`.

FileStatusResponse

Ответ проверки статуса:

- `fileId` ;
- `originalFileName` ;
- `status` ;
- `convertedFilePath` ;
- `errorMessage` .

Для незавершенной задачи путь и ошибка могут быть `null` . Для ошибки заполнено `errorMessage` , для успеха — путь PDF.

ConvertedFile

Внутренний объект скачивания:

- имя файла для HTTP attachment;
- MIME type;
- `InputStreamResource` с потоком MinIO.

FileConversionRequestedEvent

Kafka-запрос конвертеру:

- `messageId` — UUID задачи в строковом виде;
- `bucket` — bucket оригинала;
- `objectKey` — ключ оригинала;
- `fileType` — сейчас передается `null` , тип определяет конвертер;
- `originalFileName` — имя для определения расширения и результата.

FileConversionCompletedEvent

Успешный ответ конвертера. Содержит идентификаторы, координаты оригинала и PDF, а также тип файла. `flow-manager` использует главным образом `messageId` , `pdfBucket` и `pdfObjectKey` .

FileConversionErrorEvent

Ответ об ошибке:

- `fileId` — UUID задачи;
- `errorMessage` — причина сбоя.

ErrorResponse

Единый HTTP-контракт ошибки с полями `code` и `message` .

8. Пакет `service`

Пакет содержит интерфейсы, отделяющие бизнес-сценарии от инфраструктуры.

FileFlowService

Главный прикладной контракт:

- `upload` — принять файл и создать асинхронную задачу;
- `getStatus` — получить состояние;
- `downloadConvertedFile` — получить готовый PDF.

FileTaskService

Управляет сущностью задачи:

- создание `PROCESSING` ;
- поиск по UUID;
- переход в `SUCCESS` ;
- переход в `ERROR` .

OutboxService

Управляет надежной очередью событий в PostgreSQL:

- сохраняет запрос конвертации;
- выбирает сообщения, готовые к публикации.

StorageService

Абстракция объектного хранилища:

- загрузка оригинала;
- скачивание по bucket и key;
- проверка существования;
- удаление оригинала при компенсации.

Интерфейсы уменьшают связанность и позволяют тестировать прикладную логику без реальных Kafka, MinIO и БД.

9. Пакет `service.impl`

FileFlowServiceImpl

Главный координатор всего процесса.

Загрузка файла

Метод `upload` :

1. Генерирует временный `storageId` .
2. Определяет исходное имя файла.
3. Загружает оригинал в MinIO.
4. Запускает транзакцию PostgreSQL через `TransactionTemplate` .
5. Создает `FileTask` в статусе `PROCESSING` .

6. Создает outbox-сообщение `FileConversionRequestedEvent` .

7. Возвращает UUID задачи и статус.

Задача и outbox-событие сохраняются одной БД-транзакцией. Поэтому ситуация «задача создана, но Kafka-запрос забыт» не возникает.

MinIO не входит в эту транзакцию. Если сохранение задачи или outbox завершается ошибкой, catch-блок удаляет ранее загруженный объект. Это компенсационная операция.

Получение статуса

`getStatus` находит `FileTask` и преобразует ее в `FileStatusResponse` .

Скачивание

`downloadConvertedFile` разрешает скачивание только при `SUCCESS` . Иначе выбрасывается `FileNotReadyException` . Затем проверяются bucket и key результата, открывается поток MinIO и формируется имя PDF.

Имя скачивания строится из исходного имени с заменой расширения на `.pdf` .

`FileTaskServiceImpl`

Работает с `FileTaskRepository` в транзакциях.

`createProcessingTask` создает сущность с именем оригинала, путем MinIO и статусом `PROCESSING` .

`findById` возвращает задачу или выбрасывает `FileNotFoundException` .

`markSuccess` устанавливает `SUCCESS` , сохраняет bucket и key конвертированного файла, очищает старую ошибку.

`markError` устанавливает `ERROR` и сохраняет текст ошибки.

`OutboxServiceImpl`

`enqueueFileConversionRequested` сериализует событие через Jackson и создает `OutboxMessage` с topic из `KafkaProperties` . Aggregate ID и message key равны UUID задачи.

`findReadyForPublishing` делегирует repository выбор готовой пачки сообщений.

Ошибка JSON-сериализации оборачивается в `KafkaPublishException` , хотя реальной отправки в Kafka на этом этапе еще нет.

`MinioStorageService`

В `@PostConstruct` вызывает `ensureBucketExists` : проверяет bucket и создает его при отсутствии.

`uploadOriginal` формирует объект вида `original/{fileId}/{normalizedName}` и загружает поток multipart-файла с размером и content type.

`download` получает объект из указанного bucket.

`exists` проверяет объект в настроенном основном bucket через `statObject` .

`delete` удаляет объект и используется для компенсации неуспешной транзакции создания задачи.

Все ошибки SDK преобразуются в `StorageException` .

10. Пакет `entity`

`FileStatus`

Состояния бизнес-задачи:

```
PROCESSING -> SUCCESS
           -> ERROR
```

`FileTask`

JPA-сущность таблицы `file_tasks` :

- `id` — UUID задачи;
- `originalFileName` — пользовательское имя;
- `originalMinioPath` — key оригинала;
- `convertedMinioBucket` — bucket PDF;
- `convertedMinioPath` — key PDF;
- `status` ;
- `errorMessage` ;
- `createdAt` , `updatedAt` .

Новая задача создается в `PROCESSING` . Сущность хранит полный результат асинхронного процесса, необходимый для `status` и `download API`.

`OutboxStatus`

- `PENDING` — ожидает публикации или повторной попытки;
- `SENT` — Kafka подтвердила сообщение;
- `FAILED` — исчерпано максимальное число попыток.

`OutboxMessage`

JPA-сущность надежной очереди:

- UUID сообщения;
- aggregate ID задачи;
- topic и Kafka key;
- JSON payload;
- `status`;
- число попыток;
- время следующей попытки;
- последняя ошибка;
- временные метки.

Конструктор создает `PENDING` с нулем попыток и немедленной готовностью.

`markSent` устанавливает `SENT` , время отправки и очищает ошибку.

`markFailed` увеличивает счетчик. До достижения лимита сообщение остается `PENDING` с новым `nextAttemptAt`; после достижения лимита становится `FAILED`.

Ошибка обрезается до 2048 символов в соответствии со схемой БД.

11. Пакет `repository`

`FileTaskRepository`

Стандартный `JpaRepository<FileTask, UUID>`. Дополнительных запросов нет: используются `save` и `findById`.

`OutboxMessageRepository`

Кроме CRUD содержит native query `findReadyForPublishing`.

Запрос выбирает `PENDING`, у которых наступил `next_attempt_at`, сортирует по времени создания и ограничивает batch size.

`FOR UPDATE SKIP LOCKED` позволяет нескольким экземплярам `flow-manager` параллельно публиковать разные сообщения. Заблокированные другим экземпляром строки пропускаются, а не заставляют всю пачку ждать.

12. Пакет `kafka`

`OutboxKafkaPublisher`

Планировщик outbox. `@Scheduled` запускает `publishPending` через заданный интервал.

Метод транзакционно получает пачку сообщений и для каждого вызывает `publish`.

`publish` отправляет строковый JSON через `KafkaTemplate`, ожидает подтверждение не дольше `sendTimeoutMs`, затем вызывает `markSent`.

При любой ошибке вызывает `markFailed`, задавая следующую попытку через `retryDelayMs`. После `maxAttempts` сообщение окончательно получает `FAILED`.

Такой подход решает временную недоступность Kafka и гарантирует, что событие остается в БД после перезапуска приложения.

Ограничение: отправка Kafka и установка `SENT` не атомарны. Если broker принял сообщение, а транзакция БД не зафиксировалась, возможна повторная отправка. Получатель должен быть идемпотентным.

`FileConversionResultConsumer`

Интерфейс обработки двух типов результата: успешного и ошибочного.

`KafkaFileConversionResultConsumer`

Содержит два `@KafkaListener`:

- `result topic`;

- error topic.

Kafka value сначала принимается как строка, затем вручную десериализуется Jackson.

Успешное событие преобразует `messageId` в UUID и вызывает `markSuccess` с bucket и key PDF.

Ошибочное событие вызывает `markError`.

Неизвестная задача и некорректный UUID журналируются, но не пробрасываются.

Offset подтверждается в `finally` даже при некорректном JSON или ошибке обработки. Это предотвращает бесконечное чтение плохого сообщения, но приводит к его окончательной потере без DLQ.

13. Пакет `config`

`KafkaProperties`

Настройки трех topic:

- conversion request;
- conversion result;
- conversion error.

`KafkaConfig`

Включает Kafka, создает producer/consumer factory, `KafkaTemplate<String, String>` и listener container с ручным ask.

Сервис намеренно передает JSON как строки, поэтому DTO не связаны с Kafka type headers.

`MinioProperties`

Содержит endpoint, access key, secret key и bucket.

`MinioConfig`

Создает общий `MinioClient`.

`OutboxProperties`

Настройки batch size, интервала планировщика, timeout отправки, задержки retry и максимума попыток.

Compact constructor заменяет некорректные значения безопасными defaults.

`SchedulingConfig`

Включает `@EnableScheduling` и регистрацию `OutboxProperties`. Благодаря этому publisher запускается автоматически.

14. Пакет `exception`

`EmptyFileException`

Пустой multipart-файл. Возвращается клиенту как `400 Bad Request`.

`FileNotFoundException`

Задача с UUID отсутствует. Возвращается `404 Not Found`.

`FileNotReadyException`

PDF еще не готов или задача завершилась ошибкой. Возвращается `409 Conflict`.

`StorageException`

Ошибка MinIO: загрузка, скачивание, проверка, создание bucket или удаление. Возвращается `502 Bad Gateway`.

`KafkaPublishException`

Ошибка подготовки/публикации Kafka-события. Возвращается `503 Service Unavailable`.

`GlobalExceptionHandler`

`@RestControllerAdvice`, переводящий исключения в `ErrorResponse` и HTTP status. Неожиданные ошибки возвращаются как `500 Internal Server Error`.

Handler также журналирует исключения. Клиент получает стабильный код ошибки вместо stack trace.

15. Схема PostgreSQL

`file_tasks`

<code>id</code>	<code>uuid PK</code>
<code>original_file_name</code>	<code>varchar(255) not null</code>
<code>original_minio_path</code>	<code>varchar(1024) not null</code>
<code>converted_minio_bucket</code>	<code>varchar(255)</code>
<code>converted_minio_path</code>	<code>varchar(1024)</code>
<code>status</code>	<code>varchar(32) not null</code>
<code>error_message</code>	<code>varchar(2048)</code>
<code>created_at</code>	<code>timestampz not null</code>
<code>updated_at</code>	<code>timestampz not null</code>

outbox_messages

id	uuid PK
aggregate_id	uuid not null
topic	varchar(255) not null
message_key	varchar(255) not null
payload	text not null
status	varchar(32) not null
attempts	int not null
next_attempt_at	timestampz not null
last_error	varchar(2048)
created_at	timestampz not null
updated_at	timestampz not null
sent_at	timestampz

Индексы оптимизируют выбор готовых outbox-сообщений и поиск по aggregate ID.

Liquibase создает схему, Hibernate работает в режиме `validate`.

16. Полный пользовательский сценарий

1. Клиент отправляет файл через REST.
2. Пустой файл отклоняется.
3. Оригинал сохраняется в MinIO.
4. В одной БД-транзакции создаются задача и outbox-сообщение.
5. Клиент сразу получает UUID и `PROCESSING`.
6. Планировщик публикует запрос в Kafka.
7. Конвертер создает PDF и публикует результат.
8. Consumer меняет задачу на `SUCCESS` или `ERROR`.
9. Клиент опрашивает status endpoint.
10. При `SUCCESS` скачивает PDF.

17. Что решает outbox pattern

Без outbox код должен был бы сначала сохранить задачу, затем отправить Kafka-сообщение. Между этими операциями приложение могло завершиться, оставив вечную задачу `PROCESSING` без запроса конвертеру.

Outbox сохраняет задачу и намерение отправить событие одной транзакцией. Даже после перезапуска publisher увидит `PENDING` и продолжит доставку.

Гарантия здесь — как минимум одна доставка. Возможные дубликаты должны отфильтровываться конвертером по `messageId`.

18. Конфигурация по умолчанию

- HTTP: 8080 ;
- PostgreSQL: localhost:5433/flow_manager ;
- Kafka: localhost:9094 ;
- MinIO API: localhost:9002 ;
- MinIO Console: localhost:9003 ;
- Eureka: localhost:8761/eureka/ ;
- размер файла и запроса: 50 MB;
- outbox batch: 50;
- публикация: каждые 5 секунд;
- retry: через 30 секунд;
- максимум попыток: 10.

19. Тесты

Тесты покрывают:

- отклонение пустого файла facade-ом;
- создание задачи и outbox при upload;
- компенсационную и прикладную логику flow service;
- поиск и переходы статусов задач;
- обработку success/error Kafka-событий;
- поведение при отсутствующей задаче;
- запрет скачивания незавершенного файла.

Не покрыты полноценные интеграционные сценарии с реальными PostgreSQL, Kafka и MinIO, publisher retry, Liquibase и HTTP API.

20. Основные достоинства

- Четкое разделение controller, facade, services и repositories.
- Асинхронный API не блокирует клиента.
- Transactional outbox защищает от потери запроса.
- Retry Kafka-публикации сохраняется между перезапусками.
- SKIP LOCKED поддерживает горизонтальное масштабирование publisher-a.
- Оригинал удаляется при сбое создания задачи.
- PDF скачивается потоково.
- Исключения преобразуются в стабильные HTTP-контракты.

21. Проблемы и риски

- Result consumer подтверждает даже поврежденные и необработанные события; DLQ отсутствует.

- Между Kafka result и обновлением `file_tasks` нет inbox/idempotency-механизма.
- Outbox допускает дубликаты после сбоя между Kafka ack и commit БД.
- После исчерпания попыток outbox становится `FAILED`, но задача остается `PROCESSING`.
- Нет автоматической сверки зависших задач и `FAILED` outbox.
- MinIO и PostgreSQL не имеют общей транзакции; компенсационное удаление тоже может завершиться ошибкой.
- `exists` проверяет только основной bucket и почти не участвует в download-сценарии.
- При `ERROR` download отвечает общей ошибкой «не готов», а не возвращает сохраненную причину.
- Нет проверки расширения, MIME type и сигнатуры файла на входе.
- Имя файла требует дополнительной защиты от опасных символов в HTTP header и object key.
- Нет авторизации: любой знающий UUID может проверить статус и скачать файл.
- UUID не является полноценной моделью контроля доступа.
- Нет Actuator, метрик, tracing и административного API повторной отправки.
- Отсутствуют правила очистки оригиналов, PDF, задач и outbox.

22. Рекомендуемые улучшения

1. Добавить DLQ для невалидных result/error сообщений.
2. Добавить inbox или идемпотентную фиксацию Kafka-результатов.
3. При окончательном `FAILED` outbox переводить связанную задачу в `ERROR`.
4. Добавить scheduler поиска зависших `PROCESSING`.
5. Добавить endpoint повторного запуска или переотправки.
6. Ввести аутентификацию и владельца задачи.
7. Валидировать формат, MIME type и сигнатуру файла.
8. Добавить lifecycle/retention для MinIO и БД.
9. Подключить Actuator, метрики outbox lag и tracing.
10. Добавить Testcontainers-интеграцию полного пути REST -> Kafka -> result -> download.